MPU-6050 TABANLI GERÇEK ZAMANLI ÖLÇÜM SİSTEMİ:

1.Amaç

STM32C092RC üzerinde tamamlayıcı filtre ile roll/pitch hesaplanması ve MATLAB görselleştirme.

2.Sistem Mimarisi

```
[MPU-6050]  --I²C2, 100 kHz-->  [STM32C092RC]  --USART2, 115200 8N1-->  [MATLAB]
   Sensör          14 bayt/okuma     Filtre + açı       3B model + grafik
```

Bu verilen görevde bir mpu6050 sensörü kullanılarak STM32C092RC kartı üzerinde gerçek zamanlı bir işletim sistemi olan ThreadX (RTOS mimarisi çeşidi) ile ölçümü ve görselleştirmesini yapmaya çalıştım. Sensörden 100Hz hızında alınan verileri I2C hattı üzerinden mikrodenetleyiciye ordan complementary filter ile birleştirerek elde edilen verileri UART üzerinden CSV formatında bilgisayara aktardım.

Sistem; tek başına kullanıldığında gürültülü olan ivmeölçer ile zamanla sürüklenen (drift) jiroskobun zayıf yönlerini birbiriyle telafi ederek, hesaplama yükü düşük fakat kararlı bir açı sağlamakta. Test ölçümlerini alt kısımlara ekledim.

Kod kısmına geçmeden tüm ayarlarımı CubeMX ortamında ayarlaşmış olup RTOS işletim sisteminide ordan seçtim. FreeRTOS kullanmını çok yaygın olmasına ragmen ben bu kartta default olarak gelmediği için sonradan kurup karmaşıklık yaratma ihtimali nedeniyle kartta default gelen ThreadX arayüzünü kullandım. Bu nedenle tercih ettim. Bahsi geçen ayarları alt kısımdan inceleyebilirsiniz.

Diğer ödevimden sensor verisi alma ve UART gönderme işlemlerine hakimdim ve orda açıklamıştım bu nedenle o kısımlara çok fazla değinmiyorum. Bu ödevde ise benim için en önemli olan iş filtreleme ve matematikti.

3. Filtre ve Matematiği

Sensörden alınan veriler ham veriler olduğu için bunlarla işlem yapmak çok sakıncalıdır. Yukarda da bahsettiğim gibi ivmeölçer kısa vadede çok gürültü sorunları ile jiroskop ise uzun vade driftleri ile bize sıkıntı çıkartır. Bu sorunları aşmak amacıyla kodumuza complementary filter (tamamlayıcı filtre) kuruldu. Bu filtrenin temeldeki amacı iki ayrı değerin sapmalarını birbirlerine oranlayarak birbirlerinin hatalarını kapamaktır. Yani kısa vadede ivmeölçerin gürültülü yapısını jiroskopa daha çok güvenerek aşabiliyoruz. Kalman Filtresine gibi matris işlemleri içermediği için benimki gibi FPU (floating point unit) içermeyen cortex-m0+ işlemcilerde görece hızlı sürelerde çalışabilir.

```
r_acc  = atan2(ay, az) * 180/pi                    // ivmeölçerden roll
p_acc  = atan2(-ax, sqrt(ay^2 + az^2)) * 180/pi    // ivmeölçerden pitch
açı    = a * (açı_önceki + omega * dt) + (1 - a) * açı_ivme    // a = alfa
```

Burada $\alpha = 0{,}98$ ve $dt = 0{,}01$ s. Filtrenin davranışı zaman sabiti ile karakterize edilir:

$$\tau = \alpha \cdot dt / (1 - \alpha) = 0{,}98 \times 0{,}01 / 0{,}02 = 0{,}49 \text{ s} \quad \rightarrow \quad f_{kesim} \approx 0{,}32 \text{ Hz}$$

Yani 0,32 Hz üzerindeki hareketlerde jiroskop, altındaki yavaş değişimlerde ise ivmeölçer baskındır. α büyütülürse sistem daha pürüzsüz fakat daha geç tepki veren, küçültülürse daha hızlı fakat daha gürültülü bir hâle gelir. 0,98 değeri bu iki uç arasındaki dengeyi temsil eder.

-Açı Sıçraması Sorunu

Filtreyi uygulayıp ilk testleri aldığımda farkettiğim en önemli sıkıntılardan birisi sensörü tam zıt eksende 180 derece çevirdiğim zaman düz akıcı olarak değil sıçrama yapıp ters yönden dönmesiydi. Bu sorunu araştırdığımda roll açısının -180 ile +180 arasında tanımlandığını ve sınır geçişlerinde mesela +181 olacağı zaman aniden -179 olmasıydı. Bu sorunun adı Wraparound Etkisi'imiş. Çözüm olarak kodumda Roll değeri hesaplamasını değiştirmekte buldum. Yazılıma iki değer arasında (179'a -179 olduğunu varsayalım.) 358 derece fark yok sadece sınır çizgisine 2 derece olduğunu yazmam lazımdı. Bunuda aşağıdaki kod ile çözdüm.

```
/* --- TAMAMLAYICI FILTRE : ROLL */
   float r_pred = roll + fgx * dt;         /* jiroskop tahmini     */
   float err    = r_acc - r_pred;          /* ivmeolcer duzeltmesi  */
   while (err >  180.0f) err -= 360.0f;
   while (err < -180.0f) err += 360.0f;
   roll = r_pred + ALPHA * err;
   if (roll >  180.0f) roll -= 360.0f;
   if (roll < -180.0f) roll += 360.0f;
```

Pitch değerlerinde böyle bir sıkıntı görmediğim için onu düz olarak

```
/* --- TAMAMLAYICI FILTRE : PITCH  */
   pitch = (1.0f - ALPHA) * (pitch + fgy * dt) + ALPHA * p_acc;
```

bu şekilde yazdım.

-Drift Sorunu

Buraya kadar işlerin çok düzgün olduğunu düşünüp kodumu bitirdim ve takip etmeye başladım tekrardan ancak kodumda büyük bir sıkıntı olduğunu filtreden geçen verilerin özellikle roll değerinde gerçek açı değerini bulmak için gecikmeli şekilde gelmesiydi. Başta bunun kullandığım kart ile ilgili olduğunu düşünüp kartı değiştirdim daha güçlü olan stm32f4 serisi bir kartta aynı kodu çalıştırdım ancak değerleri aynı görünce sıkıntıtın kartta olmadığını anladım.

Bu seferde sıkıntıtın kaynağının RTOS mimarisinde olabileceğini düşündüm çünkü tasklar arasındaki süreleri ve özellikle veri okuma hızını 100hz (10ms) ben belirlemiştim. Sonrasında bu ödevi superloop içinde klasik yöntemlerle çalıştırdım ve sonunda hatanın burdan kaynakladığını anladım çünkü superloop'ta çalışırken bir açı kayması yoktu.

Sorun benim RTOS'ta iletim süresi ve veri okuma sürelerini hesaba katmadan 10ms süreyi ayarlamış olmamdı veri iletilirken bazen bu 10ms sınırı aşılıp eksik veri geliyordu zamanla veri güncellendikçe gerçek değerine oturmaya başlıyordu.

ÇÖZÜM:

Buna çözüm olarak araştırma yaptığımda ThreadX'te semaphore mantığını gördüm. Semaphore bir kaynağın müsait olup olmadığını tutan bir sayaçtır diyebiliriz. Bir task semaphore almak istediğinde semaphore'un değeri 0 dan büyükse değer 1 azaltılır ve task çalışmaya devam eder ancak 0 ise değeri o zaman semaphore serbest Kalana kadar task Suspended (bekleme) modunda kalır.

Bu mantığı koduma uyguladıktan sonrasında bu sorunu da çözmüş oldum çünkü tasklar artık birbirleri ile bir nevi haberleşmiş ve birbirlerinin başlangıç ve bitişlerini takip edebilir hale gelmişlerdi.

Ve burda dikkat edilmesi gereken şeylerden birisi de sorunlu kodda execution time sürekli değiştikçe dt değerini de değiştiriyodu. Bu bizim roll değerini hesaplarken integral almada kullandığımız için zamanla sürekli birikiyo ve drifti arttırıyodu. Bu kodda da artık semaphore mantığı ile kurgulandığı için dt yi değişken değil sabit değer olarak 0.01 atamak drifti büyük ölçüde azalttı.

Ayrıca buna ek olarak koduma 10sn lik bir başlangıç kalibrasyonu ekledim. 10sn boyunca gelen değerli topluyo ve bunların ortalamasını alarak bize daha sabit bir veri gelmesini sağlıyor ilk anda.

4. MATLAB Görselleştirmesine Geçiş

En sonunda artık verileri RTOS mimarisinde doğru alıp takip ettikten sonra matlab üzerinde görselleştirme kısmına geçtim. Burda da karşılaştığım problem aslında verileri float olarak göndermemden dolayı matlabın arka planda o matematiği 100hz hızında anlık olarak yaptığı için takılma sorunuydu. Özellikle 10 20sn den sonra takılma iyice artıyodu.

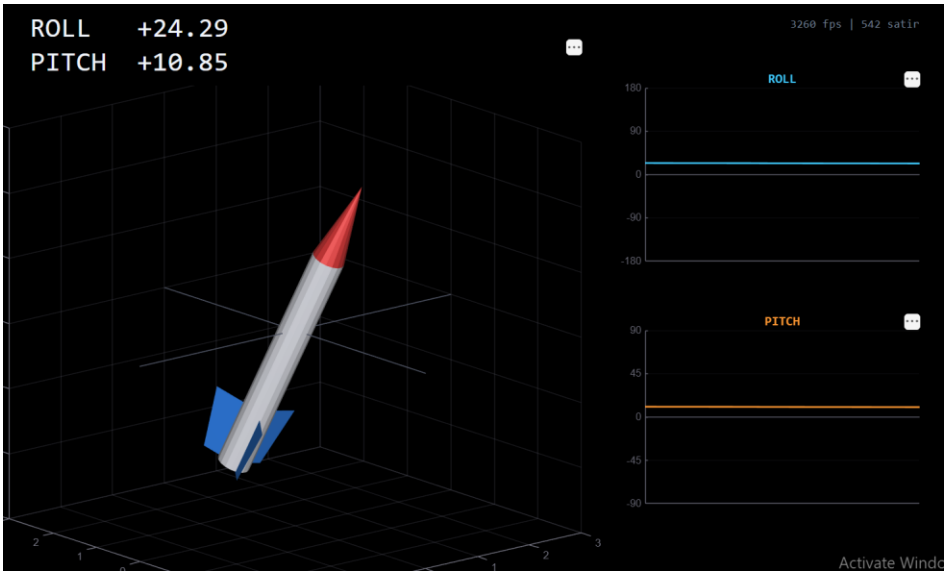
Bunu çözmek adına kartımın her döngüde CSV formatında matlab'a gönderdiği şu paketi:

```
ax, ay, az, gx, gy, gz, roll, pitch -eski
```

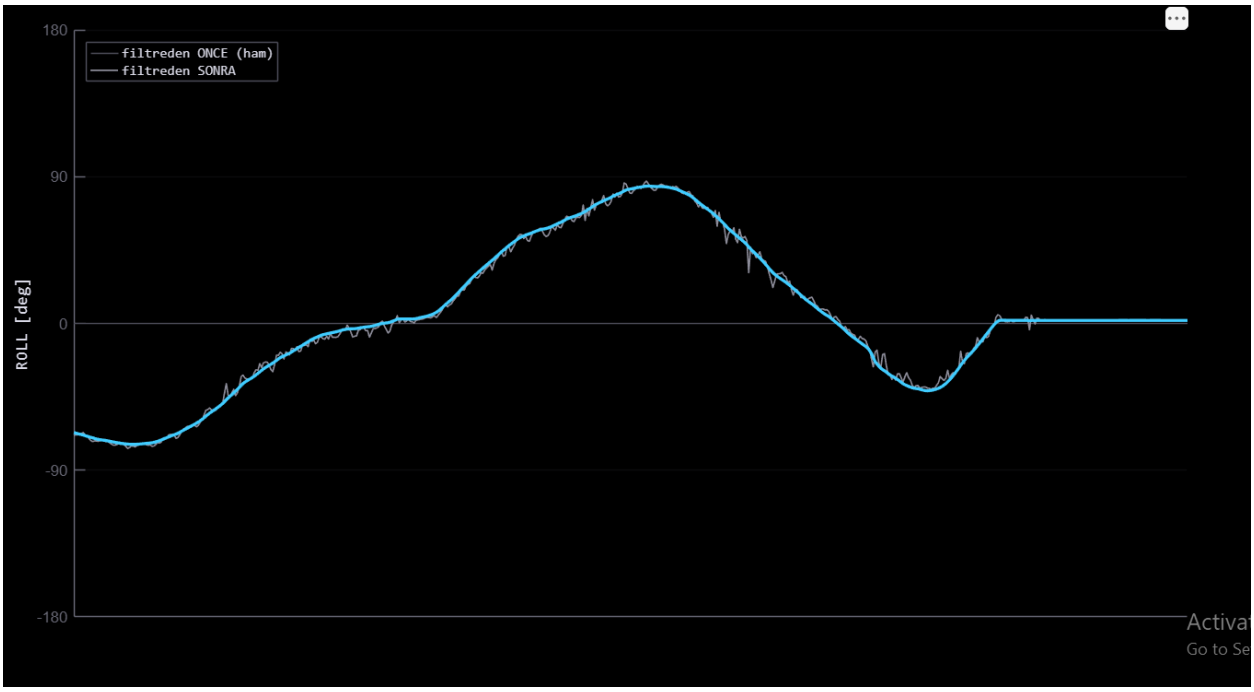
Roll ve pitch değerlerini x100 yaparak tamsayı olarak gönderdim

```
ax, ay, az, gx, gy, gz, roll × 100, pitch × 100 -yeni
```

Bu sayede hem matlabda daha akıcı hemde gömülü tarafında da daha hızlı çalışan bir kod elde ettim.



Bu şekilde anlık roll ve pitch açılarını da yan tarafa basit bir grafik olarak ekledim.

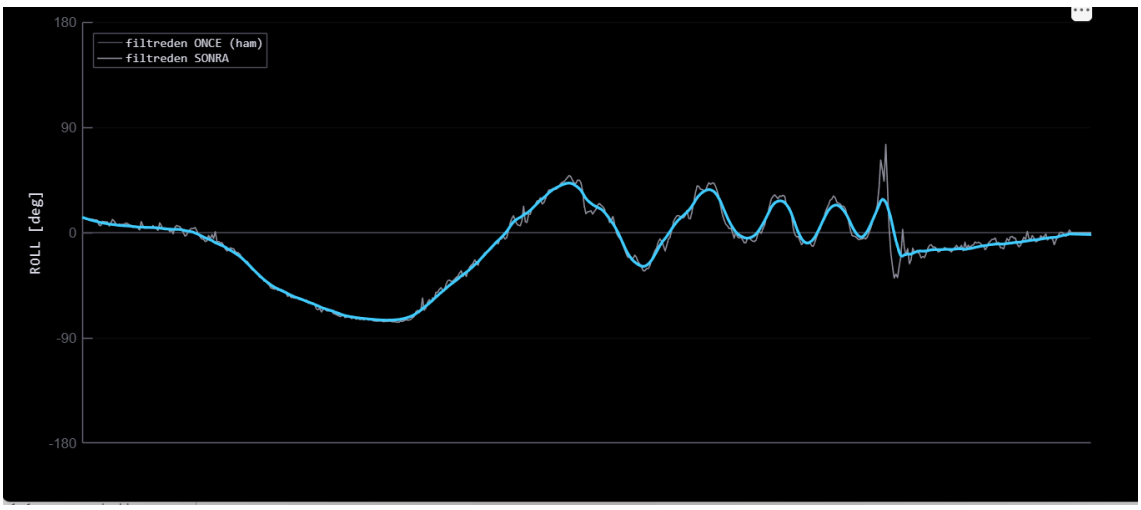


Sonrasında matlabda bir kod daha yazarak filtrenin ham veriye göre farkını roll açısı üzerinden görebileceğim bir grafik daha çizdirdim. Grafikte görüldüğü gibi filtre sonrasında gürültülerden arınmış anlık değişimlerde daha kararlı bir açı değeri ortaya çıktı.

Sonuç olarak sistemi RTOS mimarisi üzerinde başarılı gürültülerden uzak ve en önemlisi akıcı şekilde kurmayı başardım. Bu özellikle bana sensörü sadece okumaktansa filtreleme anlamlı veri oluşturma ve optimizasyon konusunda çok şey kattı. Tabiki gerçek ortamlarda çok daha fazla değişken bu açıların sapmasına yol açacaktır ancak en azından basit seviyede de olsa sensör verisi işleme konusunda beni birçok hataya maruz bırakarak onlara çözüm üretmemi sağladı bu proje.

İleri seviyede filtre tipi Kalman olarak değiştirilebilir bu sayede kalmanda oluşan hata matrisleri sayesinde bizim elle verdiğimiz 0.98 gibi hangi filtreden ne kadar veri alacağımız burda matris hesaplarıyla otomatik hale gelip daha iyi bir sonuç getirebilir. Ancak bununda daha güçlü işlemciye sahip kartlarda yapılması aradaki farkı çok daha bariz şekilde belli edecektir.

KALMANA GEÇİŞ:



Yukarıdaki fotoğrafta Kalman filtresini koduma uyguladıktan sonra ham veri ve Kalman sonrası veriyi aynı grafikte çizdirdim matlaba. Sonuç olarak baktığımızda sabit ve yavaş değişen hamlelerde complementaryden çok aşırı farkı olmamasına karşın kartı ani hareket değişimlerine soktuğumuz zaman burda değişen K (Kalman gain eski 0.02 değeri) katsayısı sayesinde anlık olarak Kalman gaini değiştirdiği için daha iyi sonuç alıyoruz.

Sensör Parametlereleri:

Sonrasında özellikle kendi merakım sonucunda bu ödevin ne alanlarda nasıl kullanılacağı ile ilgili araştırma yaptım. Elimdeki bu ödev aslında bir “attitude estimation” çekirdeği. Üstüne daha farklı katmanlar ekleyerek bir uçuş bilgisayarı halini alabilir. Ama burda en belirleyici konu benim sensörümün giriş seviyesi ucuz olduğu için birçok parametresinin düşük kalmasıydı. Bunlardan bazı hataları kalibre etmek mümkün değildir sadece ölçüm yapabiliriz (örn. bias instability , Angle Random Walk (ARW) gibi.). Bazı hataları ise kalibrasyon ile düzeltmek mümkün özellikle 6-pozition static calibration uygulaması hem kolay hem de gerçekten verilere büyük katkısı olan bir kalibrasyon.

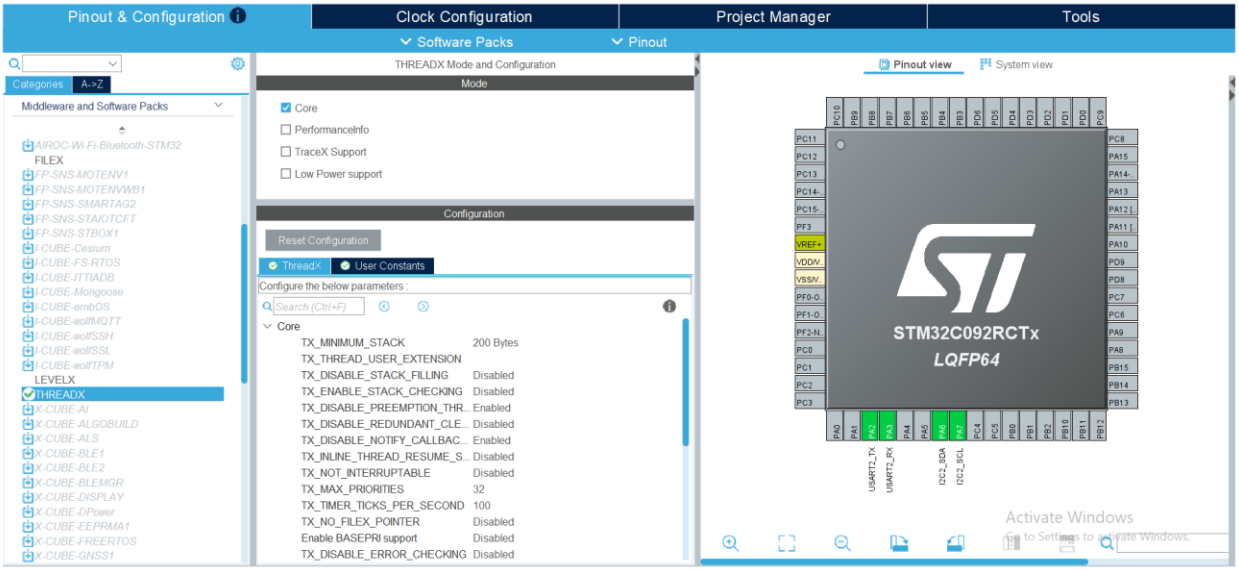
Bunu şöyle açıklayabilirim biz sensörün ham değerlerini açıya çevirmek için sabit bir sayı kullanıyoruz (örn. 16834) bu değer sayesinde ivmeölçerden gelen değeri bu sayıya bölünce 1g olduğunu kabul ediyoruz. Ancak biz bu ham değerleri sensörü 6 eksende farklı farklı ölçüp toplamlarının ortalamasını alırsak gerçekte bu sensörün bölünmesi gereken değeri elde ederiz. Çünkü sensör hareketsizken ona etki eden tek kuvvet yerçekimi olur. Bu sayede ivmeölçerin kendi hatasını telafi edip gerçek değerlere çok daha yaklaşmış oluruz.

```
/* ============================================================================
 *  IVMEOLCER KALIBRASYON KATSAYILARI
 *                 6-pozisyon testi
 *
 *    offset = (pozitif_olcum + negatif_olcum) / 2      [LSB]
 *    scale  = (pozitif_olcum - negatif_olcum) / 2      [LSB/g]
 * ============================================================================*/
#define ACC_OFF_X       651.5f
#define ACC_OFF_Y       415.5f
#define ACC_OFF_Z       800.5f

#define ACC_SCL_X       16359.5f
#define ACC_SCL_Y       16393.5f
#define ACC_SCL_Z       16863.5f
```

Burda görüldüğü gibi değerleri hesapladıktan sonra kodumuzda yerine yazdım. Ve gerçeğe çok daha yakın bir açı elde etmiş olduk.

CubeMX Ayarları:



Debug Sonuçları:

Expression	Type	Value
dbg_roll	volatile float	2.61710215
dbg_pitch	volatile float	-1.61310685
dbg_roll_acc	volatile float	2.6593945
dbg_pitch_acc	volatile float	-1.37708211
dbg_sample_cnt	volatile uint32_t	991
Add new expression		

STM32C092RC Kodları:

```
/******************************************************************************
*******************************************************************************
* @file    app_threadx.c
* @brief   MPU-6050 + Complementary Filter + ThreadX
```

```
 *        STM32C092RC (Cortex-M0+)
 *        Göktan.sarıkan
 ******************************************************************************
 ******************************************************************************
 */

/* Includes ------------------------------------------------------------------*/
#include "app_threadx.h"
#include <math.h>
#include <stdio.h>
#include <string.h>
#include "main.h"

/* Private define ------------------------------------------------------------*/
/* --- MPU-6050 --- */
#define MPU_ADDR      (0x68 << 1)   /* HAL 8-bit adres ister     */
#define MPU_REG_DATA    0x3B        /* ACC_XOUT_H (14 byte blok)  */
#define MPU_REG_WHOAMI  0x75
#define APP_PROVIDES_KERNEL_ENTRY 1

/* --- Olcek katsayilari --- */
#define GYRO_LSB       65.5f        /* +-500 dps  -> LSB/(degree/s)  */
#define ACC_LSB        16384.0f     /* +-2 g      -> LSB/g       */
#define RAD2DEG        57.29578f

/* --- Tamamlayici filtre --- */
#define ALPHA          0.02f        /* accelerometer katkısı (%2)  */

/* --- Zamanlama --- */
#define SAMPLE_TICKS    1           /* 1 ThreadX tick = 10 ms -> 100 Hz */
#define CALIB_SAMPLES   1000        /* 1000 x 10 ms = 10 saniye    */

/* ms -> tick cevirici (tick frekansi degisirse kod bozulmasin diye) */
#define MS_TO_TICK(ms)  (((ms) * TX_TIMER_TICKS_PER_SECOND) / 1000)

/* --- Thread ayarlari --- */
#define IMU_STACK_SIZE   1024
#define UART_STACK_SIZE  1024
#define IMU_PRIORITY     5      /*---> Düşük sayı high priority ----- */
#define UART_PRIORITY    10
#define QUEUE_DEPTH      8
```

```
#define APP_POOL_SIZE 4096


/* Private typedef -----------------------------------------------------------*/
/* Sensorden okunan ham degerler */
typedef struct
{
  int16_t ax, ay, az;
  int16_t gx, gy, gz;
} imu_raw_t; /* IMU dan gelen 16 bayt ham veri paketi */

/* Kuyrukta tasinan mesaj: 8 x int16 = 16 byte = 4 ULONG (TX_4_ULONG) */
typedef struct
{
  int16_t ax, ay, az;
  int16_t gx, gy, gz;
  int16_t roll100;            /* degree x 100 */
  int16_t pitch100;           /* degree x 100 */
} imu_msg_t;




/* Private variables ---------------------------------------------------------*/

extern I2C_HandleTypeDef  hi2c2;
extern UART_HandleTypeDef huart2;

/* ThreadX nesneleri */
static TX_THREAD    imu_thread;     /* sensor okuma + filtre       */
static TX_THREAD    uart_thread;    /* CSV gonderme                */
static TX_QUEUE     imu_queue;      /* IMU -> UART veri yolu       */
static TX_TIMER     sample_timer;   /* 100 Hz periyot uretici      */
static TX_SEMAPHORE sample_sem;     /* "yeni ornek zamani" isareti  */
static TX_SEMAPHORE uart_tx_done;   /* "UART gonderimi bitti"      */

/* LIVE EXPRESSIONS IZLEME */
volatile float dbg_roll = 0.0f;                 /* Roll Açısı*/
volatile float dbg_pitch = 0.0f;                /* Pitch Açısı */
volatile float dbg_roll_acc = 0.0f;             /* Ham Roll Açısı*/
volatile float dbg_pitch_acc = 0.0f;            /* Ham Pitch Açısı*/
volatile float dbg_gyro_x_dps = 0.0f;
```

```
volatile float dbg_gyro_y_dps = 0.0f;
volatile float dbg_bias_x = 0.0f;
volatile float dbg_bias_y = 0.0f;
volatile int16_t dbg_ax=0 , dbg_ay = 0 , dbg_az = 0;
volatile uint8_t dbg_who = 0;
volatile uint32_t dbg_sample_cnt = 0;

/* Private function prototypes -----------------------------------------------*/
static void imu_thread_entry(ULONG arg);
static void uart_thread_entry(ULONG arg);
static void sample_timer_cb(ULONG arg);

static void mpu_write(uint8_t reg, uint8_t val);
static void mpu_read(imu_raw_t *r);
static void uart_send_blocking(const char *s);

/* ============================================================================
 *  1. BOLUM : MPU-6050 SURUCUSU
 * ============================================================================*/

/* Tek bir registera tek byte yaz */
static void mpu_write(uint8_t reg, uint8_t val)
{
  HAL_I2C_Mem_Write(&hi2c2, MPU_ADDR, reg, I2C_MEMADD_SIZE_8BIT, &val, 1, 100);
}

/* 0x3B'den 14 byte oku: ACC X/Y/Z, TEMP, GYRO X/Y/Z */
static void mpu_read(imu_raw_t *r)
{
  uint8_t b[14];

  HAL_I2C_Mem_Read(&hi2c2, MPU_ADDR, MPU_REG_DATA, I2C_MEMADD_SIZE_8BIT, b, 14, 100);

  r->ax = (int16_t)(b[0]  << 8 | b[1]);
  r->ay = (int16_t)(b[2]  << 8 | b[3]);
  r->az = (int16_t)(b[4]  << 8 | b[5]);
  r->gx = (int16_t)(b[8]  << 8 | b[9]);
  r->gy = (int16_t)(b[10] << 8 | b[11]);
  r->gz = (int16_t)(b[12] << 8 | b[13]);
}
```

```
/* Sadece ACILIS sirasinda kullanilir (bkz. imu_thread_entry).
   O anda UART_Thread bos kuyrukta bekliyor, yani UART serbest. */
static void uart_send_blocking(const char *s)
{
  HAL_UART_Transmit(&huart2, (uint8_t *)s, strlen(s), 100);
}

/* ============================================================================
 *  2. BOLUM : ZAMANLAYICI  -  100 Hz periyodu uretir
 * ============================================================================*/

/*
 * Bu geri cagirma her ThreadX tick'inde (10 ms) calisir ve sadece semaforu
 * bir arttirir. Tavan (ceiling) 1 verildigi icin, IMU_Thread bir sebeple
 * gecikirse semafor birikmez -> thread "yetismek icin" pes pese kosmaya
 * calismaz, bir sonraki periyoda temiz baslar.
 */
static void sample_timer_cb(ULONG arg)
{
  (void)arg;
  tx_semaphore_ceiling_put(&sample_sem, 1);
}

/* ============================================================================
 *  3. BOLUM : IMU THREAD  -  oku, filtrele, kuyruga koy
 * ============================================================================*/
static void imu_thread_entry(ULONG arg)
{

  imu_raw_t raw;
  imu_msg_t msg;
  char      banner[64];
  uint8_t   who = 0;

  float roll = 0.0f, pitch = 0.0f;   /* filtre ciktisi [derece]  */
  float bx   = 0.0f, by    = 0.0f;   /* jiroskop bias [LSB]      */
  float dt   = (float)SAMPLE_TICKS / (float)TX_TIMER_TICKS_PER_SECOND;

  (void)arg;

  /* ---- 3.1  MPU-6050'yi baslat ---------------------------------------- */
```

```
mpu_write(0x6B, 0x80);  tx_thread_sleep(MS_TO_TICK(100));  /* reset      */
mpu_write(0x6B, 0x01);  tx_thread_sleep(MS_TO_TICK(50));   /* uyandir    */
mpu_write(0x1A, 0x03);                          /* DLPF 42 Hz  */
mpu_write(0x19, 0x09);                          /* 100 Hz      */
mpu_write(0x1B, 0x08);                          /* gyro +-500  */
mpu_write(0x1C, 0x00);                          /* acc  +-2 g  */
tx_thread_sleep(MS_TO_TICK(50));

/* ---- 3.2  Kimlik kontrolu ---------------------------------------- */
HAL_I2C_Mem_Read(&hi2c2, MPU_ADDR, MPU_REG_WHOAMI, I2C_MEMADD_SIZE_8BIT, &who, 1, 100);
sprintf(banner, "WHO_AM_I=%d (104 olmali)\r\n", who);
uart_send_blocking(banner);

/* ---- 3.3  Periyot uretecini baslat ---------------------------------- */
tx_timer_activate(&sample_timer);

/* ---- 3.4  Jiroskop bias kalibrasyonu (10 sn, karti sabit tut) ------- */
{
 long sx = 0, sy = 0;
 for (int i = 0; i < CALIB_SAMPLES; i++)
 {
   tx_semaphore_get(&sample_sem, TX_WAIT_FOREVER);   /* 10 ms bekle */
   mpu_read(&raw);
   sx += raw.gx;
   sy += raw.gy;
 }
 bx = (float)sx / (float)CALIB_SAMPLES;
 by = (float)sy / (float)CALIB_SAMPLES;
}

/* ---- 3.5  Baslangic acilari (sadece ivmeolcerden) ------------------ */
mpu_read(&raw);
roll  = atan2f(raw.ay, raw.az) * RAD2DEG;
pitch = atan2f(-raw.ax, sqrtf((float)raw.ay * raw.ay +
                        (float)raw.az * raw.az)) * RAD2DEG;

uart_send_blocking("ax,ay,az,gx,gy,gz,roll100,pitch100\r\n");

/* ---- 3.6  ANA DONGU : tam olarak 100 Hz ---------------------------- */
while (1)
{
```

```
/* Periyodun basini bekle. Thread burada CPU tuketmez. */
tx_semaphore_get(&sample_sem, TX_WAIT_FOREVER);

mpu_read(&raw);

/* Ham degerleri fiziksel birime cevir */
float fgx = (raw.gx - bx) / GYRO_LSB;    /* dps */
float fgy = (raw.gy - by) / GYRO_LSB;
float fax = raw.ax / ACC_LSB;          /* g   */
float fay = raw.ay / ACC_LSB;
float faz = raw.az / ACC_LSB;

/* Ivmeolcerden aci */
float r_acc = atan2f(fay, faz) * RAD2DEG;
float p_acc = atan2f(-fax, sqrtf(fay * fay + faz * faz)) * RAD2DEG;

/* --- TAMAMLAYICI FILTRE : ROLL */
float r_pred = roll + fgx * dt;         /* jiroskop tahmini     */
float err    = r_acc - r_pred;          /* ivmeolcer duzeltmesi  */
while (err >  180.0f) err -= 360.0f;
while (err < -180.0f) err += 360.0f;
roll = r_pred + ALPHA * err;
if (roll >  180.0f) roll -= 360.0f;
if (roll < -180.0f) roll += 360.0f;

/* --- TAMAMLAYICI FILTRE : PITCH  */
pitch = (1.0f - ALPHA) * (pitch + fgy * dt) + ALPHA * p_acc;

/* --- LIVE EXPRESSIONS TANIMLARı ---- */
dbg_roll = roll;
dbg_pitch = pitch;
dbg_roll_acc = r_acc;
dbg_pitch_acc = p_acc;
dbg_gyro_x_dps = fgx;
dbg_gyro_y_dps = fgy;
dbg_ax = raw.ax;
dbg_ay = raw.ay;
dbg_az = raw.az;
dbg_sample_cnt++;

/* --- Sonucu kuyruga birak ---------------------------------------- */
```

```
    msg.ax      = raw.ax;
    msg.ay      = raw.ay;
    msg.az      = raw.az;
    msg.gx      = raw.gx;
    msg.gy      = raw.gy;
    msg.gz      = raw.gz;
    msg.roll100  = (int16_t)(roll  * 100.0f);
    msg.pitch100 = (int16_t)(pitch * 100.0f);

    /* TX_NO_WAIT: kuyruk dolduysa (UART yetisemiyorsa) bu ornegi at.
       Orneklemeyi ASLA bekletme - filtre zamanlamasi bozulmasin. */
    tx_queue_send(&imu_queue, &msg, TX_NO_WAIT);
  }
}

/* ============================================================================
 *  4. BOLUM : UART THREAD  -  CSV satirini gonder
 * ============================================================================*/
static void uart_thread_entry(ULONG arg)
{
  imu_msg_t msg;
  char      line[80];
  int       len;

  (void)arg;

  while (1)
  {
    /* Kuyrukta mesaj yoksa burada uyur CPU harcamaz. */
    tx_queue_receive(&imu_queue, &msg, TX_WAIT_FOREVER);

    len = sprintf(line, "%d,%d,%d,%d,%d,%d,%d,%d\r\n",
                msg.ax, msg.ay, msg.az,
                msg.gx, msg.gy, msg.gz,
                msg.roll100, msg.pitch100);

    /* Interrupt ile gonder: CPU bosa donmez.
       'line' thread'in yiginda; asagida semaforu bekledigimiz icin
       gonderim bitene kadar gecerli kalir. */
    if (HAL_UART_Transmit_IT(&huart2, (uint8_t *)line, len) == HAL_OK)
    {
```

```
    tx_semaphore_get(&uart_tx_done, MS_TO_TICK(200));  /* takilirsa kurtul */
   }
  }
}

/* Gonderim bitince USART2 kesmesinden cagrilir -> UART_Thread'i uyandirir */
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
  if (huart->Instance == USART2)
  {
    tx_semaphore_put(&uart_tx_done);
  }
}









/* ==========================================================================
 *  5. BOLUM : KURULUM  -  tum ThreadX nesneleri burada olusturulur
 * ==========================================================================*/
/**
  * @brief  ThreadX uygulama nesnelerini olusturur.
  * @param  memory_ptr: CubeMX'in olusturdugu byte pool
  * @retval TX_SUCCESS veya hata kodu
  * @note   Bu fonksiyon zamanlayici (scheduler) baslamadan once calisir.
  *         Icinde HAL cagrisi YAPILMAZ (interrupt kapali).
  */
UINT App_ThreadX_Init(VOID *memory_ptr)
{
  TX_BYTE_POOL *byte_pool = (TX_BYTE_POOL *)memory_ptr;
  CHAR *ptr;

  /* --- 5.1  Kuyruk : IMU -> UART ------------------------------------- */
  if (tx_byte_allocate(byte_pool, (VOID **)&ptr,
                       QUEUE_DEPTH * sizeof(imu_msg_t), TX_NO_WAIT) != TX_SUCCESS)
  {
    return TX_POOL_ERROR;
  }
  if (tx_queue_create(&imu_queue, "IMU Queue", TX_4_ULONG,
                      ptr, QUEUE_DEPTH * sizeof(imu_msg_t)) != TX_SUCCESS)
```

```
{
  return TX_QUEUE_ERROR;
}

/* --- 5.2  Semaforlar ------------------------------------------------ */
/* Baslangic degeri 0: iki taraf da once "olay" bekler. */
if (tx_semaphore_create(&sample_sem, "Sample Tick", 0) != TX_SUCCESS)
{
  return TX_SEMAPHORE_ERROR;
}
if (tx_semaphore_create(&uart_tx_done, "UART Done", 0) != TX_SUCCESS)
{
  return TX_SEMAPHORE_ERROR;
}

/* --- 5.3  Periyot timer'i (10 ms) ---------------------------------- */
/* TX_NO_ACTIVATE: IMU_Thread sensoru kurduktan sonra kendisi baslatir. */
if (tx_timer_create(&sample_timer, "Sample Timer", sample_timer_cb, 0,
                    SAMPLE_TICKS, SAMPLE_TICKS, TX_NO_ACTIVATE) != TX_SUCCESS)
{
  return TX_TIMER_ERROR;
}

/* --- 5.4  IMU Thread (yuksek oncelik) ------------------------------- */
if (tx_byte_allocate(byte_pool, (VOID **)&ptr,
                     IMU_STACK_SIZE, TX_NO_WAIT) != TX_SUCCESS)
{
  return TX_POOL_ERROR;
}
if (tx_thread_create(&imu_thread, "IMU Thread", imu_thread_entry, 0,
                     ptr, IMU_STACK_SIZE,
                     IMU_PRIORITY, IMU_PRIORITY,
                     TX_NO_TIME_SLICE, TX_AUTO_START) != TX_SUCCESS)
{
  return TX_THREAD_ERROR;
}

/* --- 5.5  UART Thread (dusuk oncelik) ------------------------------- */
if (tx_byte_allocate(byte_pool, (VOID **)&ptr,
                     UART_STACK_SIZE, TX_NO_WAIT) != TX_SUCCESS)
{
```

```
  return TX_POOL_ERROR;
 }
 if (tx_thread_create(&uart_thread, "UART Thread", uart_thread_entry, 0,
                      ptr, UART_STACK_SIZE,
                      UART_PRIORITY, UART_PRIORITY,
                      TX_NO_TIME_SLICE, TX_AUTO_START) != TX_SUCCESS)
 {
  return TX_THREAD_ERROR;
 }

 return TX_SUCCESS;
}

/*
 * ========================================================
 * ========================================================
            6. BOLUM -Çekirdek girişi
 * ========================================================
 * ========================================================
 */

#if APP_PROVIDES_KERNEL_ENTRY

static UCHAR
app_pool_mem[APP_POOL_SIZE];
static TX_BYTE_POOL app_pool;

VOID tx_applications_define(VOID*first_unused_memory){

            (void)first_unused_memory;

            tx_byte_pool_create(&app_pool, "APP Pool" , app_pool_mem , APP_POOL_SIZE);
            App_ThreadX_Init(&app_pool);
}

void MX_ThreadX_Init(void){

tx_kernel_enter();

}
```

```
#endif
```